Relatório de Aula — Paradigmas de Linguagens de Programação

Matheus De Oliveira Mascarello — RA 241207402

Introdução

Nessa aula a gente viu como as linguagens de programação avaliam expressões, fazem atribuições e controlam o fluxo de execução. O professor trouxe dois casos reais que deixaram bem claro por que isso importa na prática.

O primeiro foi o bug "goto fail" da Apple em 2014, onde uma linha duplicada de `goto fail;` fazia o iOS aceitar certificados falsos — ou seja, uma linha a mais no código abria uma brecha de segurança enorme. O segundo foi o caso do Zune da Microsoft, que travou no mundo inteiro no último dia de 2008 porque um laço `while` entrava em loop infinito no dia 366 de anos bissextos. Esses exemplos mostraram que entender expressões e estruturas de controle não é só teoria — é o que separa um código confiável de um desastre em produção.

1. Quiz de Abertura — Palpites antes de executar

Antes de começar a explicar, o professor pediu pra anotar um palpite sobre o que cada trecho de código imprimiria, sem executar.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

Palpite: -4 e 8 → Real: -4 e 512

2 · Java

```
1 + 2 + "3" + 4 + 5
```

Palpite: "12345" → Real: "3345"

3 · JavaScript

```
"5" - 2  e  "5" + 2
```

Palpite: 3 e 7 → Real: 3 e "52"

4 · C

```
3 > 2 > 1
```

Palpite: 1 → Real: 0

5 · Python

```
x=0; x or "padrão"  e  3>2>1
```

Palpite: 0 e True → Real: "padrão" e True

6 · JavaScript

`0=="" / ""=="0" / 0=="0"`

Palpite: false, false, true → Real: true, false, true

Ao longo da aula fomos conferindo os resultados reais e entendendo por que a maioria dos palpites estava errada. Isso foi bem útil pra fixar os conceitos.

2. Expressões Aritméticas — Precedência e Associatividade

Aprendi que cada linguagem tem suas próprias regras de precedência e associatividade, e isso pode mudar completamente o resultado de uma expressão. Por exemplo, em Python o operador ** tem precedência maior que o menos unário, então `-2 ** 2` dá -4 e não 4. Já em JavaScript, isso é erro de sintaxe — a linguagem obriga a usar parênteses para deixar a intenção clara.

Outro ponto interessante foi a questão da ordem dos operandos. Em C, a ordem de avaliação não é garantida pelo padrão, então uma função com efeito colateral pode dar resultados diferentes dependendo do compilador. Java resolve isso fixando a avaliação da esquerda para a direita. Isso me fez entender por que linguagens funcionais, que não têm efeitos colaterais, são mais previsíveis nesse sentido.

3. Conversões de Tipos e Coerção Implícita

Esse foi um dos temas que mais me chamou atenção. A coerção implícita, quando a linguagem converte tipos automaticamente sem avisar, pode esconder bugs sérios. Em C, por exemplo, `int i = 3.7;` simplesmente vira 3 em silêncio — sem erro, sem aviso.

O caso mais crítico foi o do JavaScript e PHP. Em JavaScript, `"5" - 2` dá 3 porque o `-` só existe para números, mas `"5" + 2` dá "52" porque o + prefere concatenar. E o == do JavaScript não é transitivo: `0 == ""` é true, `"" == "0"` é false, mas `0 == "0"` é true. A regra prática é sempre usar === nesses casos.

Em PHP isso chegou a causar falhas de segurança reais: hashes MD5 que começavam com `0e` eram tratados como notação científica, fazendo dois hashes diferentes serem considerados iguais com ==. Sistemas de login que comparavam hashes assim aceitavam senhas erradas.

4. Expressões Booleanas e Curto-Circuito

Aprendi que linguagens como C, Python e JavaScript aceitam qualquer valor em uma condição — 0, "", [] e None contam como falso. Já Java, Go e Rust exigem um booleano de verdade, o que evita uma classe inteira de bugs.

O curto-circuito foi outro ponto importante. Em `(a >= 0) && (b < 10)`, se a < 0 o segundo operando nem é avaliado. Isso é útil para proteger acessos a listas e ponteiros nulos, mas

perigoso combinado com efeitos colaterais. O operador `??` do JavaScript também foi apresentado como alternativa ao `||` quando o valor 0 é válido e não deve ser substituído.

5. Sentenças de Atribuição

Em C e JavaScript, a atribuição é uma expressão — ela devolve um valor. Isso permite construções como `while ((ch = getchar()) != EOF)`, que é bem prática, mas também abre espaço para o bug clássico de escrever `if (x = y)` em vez de `if (x == y)`. Python resolve isso não permitindo que a atribuição seja uma expressão, exceto pelo operador morsa `:=`.

Python também permite atribuição múltipla, como `a, b = b, a` para trocar duas variáveis sem precisar de uma temporária. Isso é bem mais legível do que a versão em C.

6. Estruturas de Controle

Böhm e Jacopini provaram que qualquer algoritmo pode ser escrito com apenas três estruturas: sequência, seleção e iteração. O goto não é necessário — mas as linguagens têm várias outras estruturas para facilitar a escrita.

Seleção: O problema do else pendente mostrou como a endentação pode enganar. Em C, o else sempre pertence ao if mais próximo, ignorando a endentação. Go e Rust resolveram isso tornando as chaves obrigatórias. O `switch` de C sem `break` é outra armadilha — o código cai nos casos seguintes sem avisar. O Java 14+ e o Rust já resolveram com `switch ->` e `match`.

Iteração: O bug do Zune ficou na memória aqui. Com `dias == 366` em um ano bissexto, nenhum ramo do while alterava a variável, causando loop infinito. Aprendi também a diferença entre `for...in` e `for...of` no JavaScript — o primeiro percorre chaves, não valores, e isso quebra qualquer soma que pareça óbvia.

Desvio incondicional: O goto foi discutido com o caso real da Apple. Uma linha `goto fail;` duplicada fazia a verificação de assinatura de certificados ser pulada sempre, aceitando certificados falsos. O `break`, `continue` e `return` são formas restritas de goto que mantêm a legibilidade sem os riscos.

7. Conclusão

A aula mostrou que entender como uma linguagem avalia expressões e controla o fluxo não é detalhe — é o que faz a diferença entre um código confiável e um que falha de formas inesperadas. As linguagens mais modernas como Go e Rust claramente aprenderam com os erros do C: exigem booleanos no if, chaves obrigatórias, conversões explícitas e `match` exaustivo. Cada uma dessas restrições elimina uma classe inteira de bugs.

O que mais ficou foi a tabela do último slide, mostrando que para cada armadilha clássica existe uma decisão de projeto que a resolve. Isso me fez ver as linguagens de uma forma diferente — não só como ferramentas, mas como projetos com escolhas que têm consequências reais.